

The Resurgence of



Perl

In this article and in the ensuing articles in this series, the author will be highlighting a resurgent Perl.

Perl was called the duct tape of the Internet by its developers. And it still holds together a lot of Net based tools – the best example being Bugzilla, the open source bug tracking tool.

With more DevOps roles popping up in the cloud space and after the release of Perl 6, this programming language is seeing a resurgence. In this article, we will gauge Perl 5 and probably look at Perl 6 in a different series. Perl 5 has one of the greatest communities around it and is growing stronger!!

Perl is an excellent tool, and you could call it the Swiss army knife of programming languages, especially when it comes to one liners. This we will see in future articles of this series, where we will use one liners to start up a Web server in Perl.

For now, let us look at setting up a Web service (if not a server) for Perl, the old fashioned way, which still runs well for products like Bugzilla.

Types of Perl Web services

Listed below are some of the common Perl Web services.

1. *CGI (Common Gateway Interface)*
 - a. `mod_cgi`
 - b. `FCGI`
 - c. `mod_perl` – This is the fastest method to implement CGI scripts and add more functionality. This is officially supported only till Apache 2.2, though it works in Apache 2.4.
2. *PSGI (Perl Web Server Gateway Interface aka its middleware)*
 - a. *Plack* – This is a set of tools for using the PSGI stack. It contains middleware components, a reference server and utilities for Web application frameworks. Plack is like Ruby's Rack or Python's Paste for WSGI– (github.com/plack/Plack)
 - b. *Web frameworks* – Examples include Catalyst, Dancer, Mojolicious, etc
 - c. *P6SGI (Server – middleware – application)*, which we will not go into too deeply in this series.

Main concerns:

- I. Setting up the middleware using Apache (Web) and CGI/ Perl (App)
- II. Trying out `mod_cgi`, and also comparing it with `mod_perl`
- III. Moving to `HTML::Template` (decouple Perl code from the `HTML/CSS/JS`)

IV. Giving PSGI a try and moving to Plack

V. Upgrading to Web frameworks

Requirements

1. A desktop or laptop (the OS is not important – even MacOS will do)
2. Internet access

If we have a GNU/Linux machine and want to set up Perl/HTML on that itself, then we could go directly to Step 3 (vi) post install instructions on Page 77, copy them to a `.sh` file and run them as executable; and instead of going to Step 3 (vii), we can go directly to Upgrade Perl section on Page 39.

If you do not have a GNU/Linux box, and want to set up a Web server and application container (in our case, the Perl module) in the cloud, that would be the main motive of this session. Future sessions would have more details on Perl+HTML. Out of all the cloud containers out there, I found AWS to be very fast and intuitive. So we will be doing our Web setup using AWS Free Tier.

Web server for Perl

1. Create an AWS Free Tier account using your email address and credit card. (It is very easy to create a free tier account; just Google for the instructions.) You do not need certification for this.
2. Before we create our instance, we need to create the following.
 - i. *A key pair*

Click on EC2 under the Compute section.

On the left pane, click on *Key Pairs* under *Network & Security*. There are two ways to do this.

(a) Let AWS create it for you.

1. Click on *Create Key Pair*.
2. Give a *Key Pair* name.
3. Hit *Create*. This will automatically create the private key (`.PEM`) and download it to your *Browser Downloads* section. Save that file and remember the path of the file.

(b) You create it and import it into AWS.

1. From Command Line on any OS, we use `ssh-keygen` to create the private and public key. On pressing enter; it will create `id_rsa(private key)` and `id_rsa.pub (public key)` files. (NOTE: AWS only supports RSA keys)
2. Using `puttygen.exe` we can save the private key (`.ppk`)